

KF, EKF, and UKF Applied to Microsoft Geolife GPS Trajectory Dataset

Trevor Gerken

Introduction

Kalman filters (KF) are used to filter noisy or uncertain measurements in real-world applications [1]. By providing filtered estimates for measures, they can provide state estimation and information for applications including autonomous robotics, signal processing, among others. The KF can also provide useful information from noisy data while utilizing relatively small computational and memory power.

In this project, various types of KF methods will be applied to the GPS trajectories in the Microsoft GeoLife dataset, which contains noisy data. Both pure filtering and state estimation KFs will be developed. The highest performing filters based on innovation and variance values for each application will then be refined and tuned further [2][3]. Once tuned, each filter will then be applied to a different trajectory to prevent overfitting.

Overview of Kalman Filters

The filtering problem is outlined in Oksendal as a time continuous linear system [4] Let's say we have observations $Z(s)$ of $Q(s)$ at times $s \leq t$. There are situations where the measurements contain a disturbed version of $Q(s)$, represented in equation 1.2.1 as follows [4].

$$Z(s) = Q(s) + \text{"noise"}$$

The Kalman filter intends to provide the best equation of $Q(t)$ that satisfies the equation above. Then, the mathematical principles of the Kalman filter are then described by defining a multi-dimensional state-space for a linear system as follows [4]

$$\text{linear system} \rightarrow dX_t = F(t)X_t dt + C(t)dU_t; F(t) \in \mathbb{R}^{n \times n}, C(t) \in \mathbb{R}^{n \times p} \quad (6.3.1)$$

$$\text{linear observation} \rightarrow dZ_t = G(t)X_t dt + D(t)dV_t; G(t) \in \mathbb{R}^{m \times n}, D(t) \in \mathbb{R}^{m \times r} \quad (6.3.2)$$

Where X_t represents the hidden signal and Z_t is the observed system. The variables $F(t)$, $C(t)$, $G(t)$, and $D(t)$ govern the drift and diffusion terms in the system and dynamics.

Then, steps provided by Oksendal to solve the multi-dimensional Kalman filter are as follows [4]:

Step 1. Z-Linear and Z-Measurable Estimates

Step 2. The Innovation Process

Step 3. The Innovation Process and Brownian Motion

Step 4. An Explicit Formula for X_t

Step 5. The Stochastic Differential Equation for $\hat{X}_t$

It is important to note the differential notation of the linear system and observations indicate a continuous form of the Kalman filter. Also, Oksendal overview of the Kalman filter is only applicable to filtering problems that use linear systems.

GeoLife GPS Trajectory Dataset Overview

The Microsoft GeoLife dataset contains thousands of different trajectories, mostly occurring in the city of Beijing [5]. The trajectories contain a mixture of transportation modes including walking, cycling, buses, cars, and more. A visual of the heat map from the dataset's documentation can be found below.

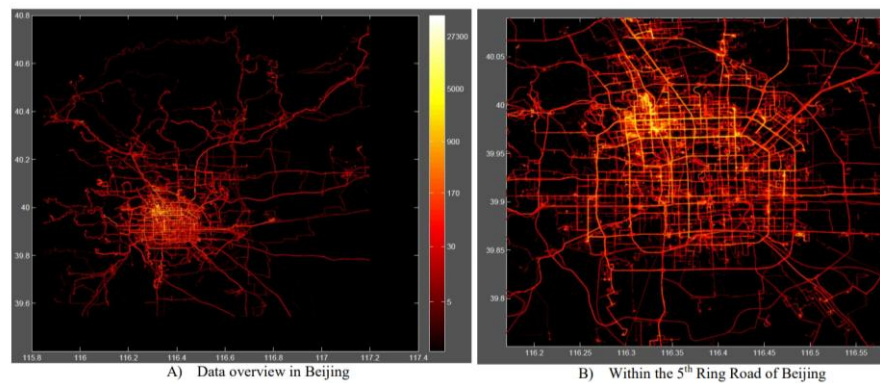
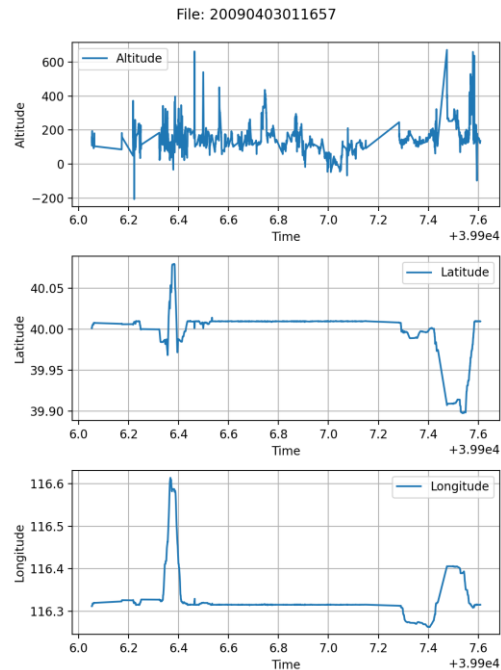


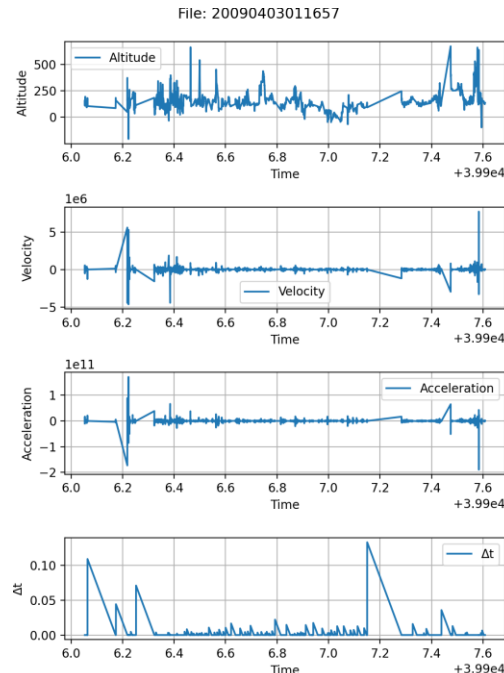
Figure 1 Distribution of the dataset in Beijing city

To test the filters, a larger trajectory file was chosen at random. The following figures outline the altitude, longitude, and latitude for the chosen trajectory file containing over 14,000 data points.

The trajectories include position data (altitude, longitude, and latitude). The altitudes for two selected trajectories are analyzed below, which includes over 14000 data points.



After reviewing the plotted data, it was determined that the noisiest measurement in the trajectory is the altitude, which led to the filtering problem being applied to this measurement only. The following plot indicates the vertical position (altitude), velocity, and acceleration. Each of which will be used as a state variables at some point throughout the project.



The last subplot above highlights the varying timesteps in the data set. There were concerns that existing libraries in Python wouldn't accommodate the varying timesteps, which was a main reason the filters were programmed from scratch. There may have been a resolution, but combined with other reasons (discussed later), it made sense to program from scratch.

Applied Discrete Time Kalman Filters

For a multi-dimensional, discrete time Kalman filter, the state-space is outlined as follows from Haykin [6]:

$$x_{k+1} = F_{k+1,k}x_k + w_k$$

$$y_k = H_kx_k + v_k$$

Where, x_{k+1} consists of the state variables and y_k contains the measure-space variables. For 2 and 3-dimensional systems, we will examine two different scenarios: pure filtering and state estimation. For the first scenario, the 2-D and 3-D state and measured variables will be as follows:

$$\begin{array}{cc} (2-D) & (3-D) \\ x_{k+1} = \begin{bmatrix} position \\ velocity \end{bmatrix} & x_{k+1} = \begin{bmatrix} position \\ velocity \\ acceleration \end{bmatrix} \\ y_k = \begin{bmatrix} position \\ velocity \end{bmatrix} & y_k = \begin{bmatrix} position \\ velocity \\ acceleration \end{bmatrix} \end{array}$$

The pure filtering scenario allows for each state variable to be measured (or calculated). In the second scenario, a measure-space variable is removed, leaving the Kalman filter to estimate the state of the unmeasured state variable.

$$\begin{array}{cc} (2-D) & (3-D) \\ x_{k+1} = \begin{bmatrix} position \\ velocity \end{bmatrix} & x_{k+1} = \begin{bmatrix} position \\ velocity \\ acceleration \end{bmatrix} \\ y_k = [position] & y_k = \begin{bmatrix} position \\ velocity \end{bmatrix} \end{array}$$

Since velocity and acceleration aren't included in the dataset, they will be measured, or calculated using the central finite difference method as follows:

$$v = \frac{\phi(i + d_\phi) - \phi(i - d_\phi)}{2d_\phi} = \frac{\lambda(i + d_\lambda) - \lambda(i - d_\lambda)}{2d_\lambda} = \frac{h(i + d_h) - h(i - d_h)}{2d_h}$$

$$a = \frac{\phi(i + d_\phi) - 2\phi(i) + \phi(i - d_\phi)}{d_\phi^2} = \frac{\lambda(i + d_\lambda) - 2\lambda(i) + \lambda(i - d_\lambda)}{d_\lambda^2} = \frac{h(i + d_h) - 2h(i) + h(i - d_h)}{d_h^2}$$

A 2-D and 3-D filter will be created for the pure filtering and state estimation scenarios using the Kalman filter, Extended Kalman filter, and Unscented Kalman filter. To compare the performance of each filter, the latency will be extracted from the algorithm, along with the innovation for each measure-space variable and variance of each state-space variable.

In the first application of the discrete Kalman filters, minimal tuning will be done. The main goal is to establish a baseline to understand the performance of the KF, UKF, and EKF. The metrics used in the baseline will aim for a magnitude of ± 25 for the innovation RMS and average variance of for the altitude. These values are realistic since the altitude measurements range from -200 to 600. With a

variation of ± 25 , the filters will produce estimates that provide useful information. Then, once the baseline results provide insight, only the best performing filter for each application will be tuned (2-D filter, 3-D state estimation, etc.).

Kalman Filters and Linear Systems: The Kalman Filter

From Haykin, the state-space model for the discrete KF is as follows [6]:

$$x_{k+1} = F_{k+1}x_k + w_k$$

$$y_k = H_kx_k + v_k$$

Where, w_k and v_k are:

- Independent
- Zero-mean
- Normal
- Covariances of Q_k and R_k , respectively.

Haykin outlines the following process for developing a KF [6]. Starting with initialization:

$$\hat{x}_0 = E[x_0]$$

$$P_0 = E[(x_0 - E[x_0])(x_0 - E[x_0])^T]$$

The iterative portion, for each $k = 1, 2, 3, \dots$:

State estimate propagation

$$\hat{x}_k^- = F_{k,k-1}\hat{x}_{k-1}^-$$

Error covariance propagation

$$P_k^- = F_{k,k-1}P_{k-1}F_{k,k-1}^T + Q_{k-1}$$

Kalman gain matrix

$$G_k = P_k^- H_k^T [H_k P_k^- H_k^T + R_k]^{-1}$$

State estimate update

$$\hat{x}_k = \hat{x}_k^- + G_k(y_k - H_k \hat{x}_k^-)$$

Error covariance update

$$P_k = (I - G_k H_k) P_k^-$$

For the 2-D filtering system:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

For the 2-D state estimation system:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, H = [1 \quad 0]$$

For the 3-D filtering system:

$$F = \begin{bmatrix} 1 & \Delta t & \frac{\Delta t^2}{2} \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

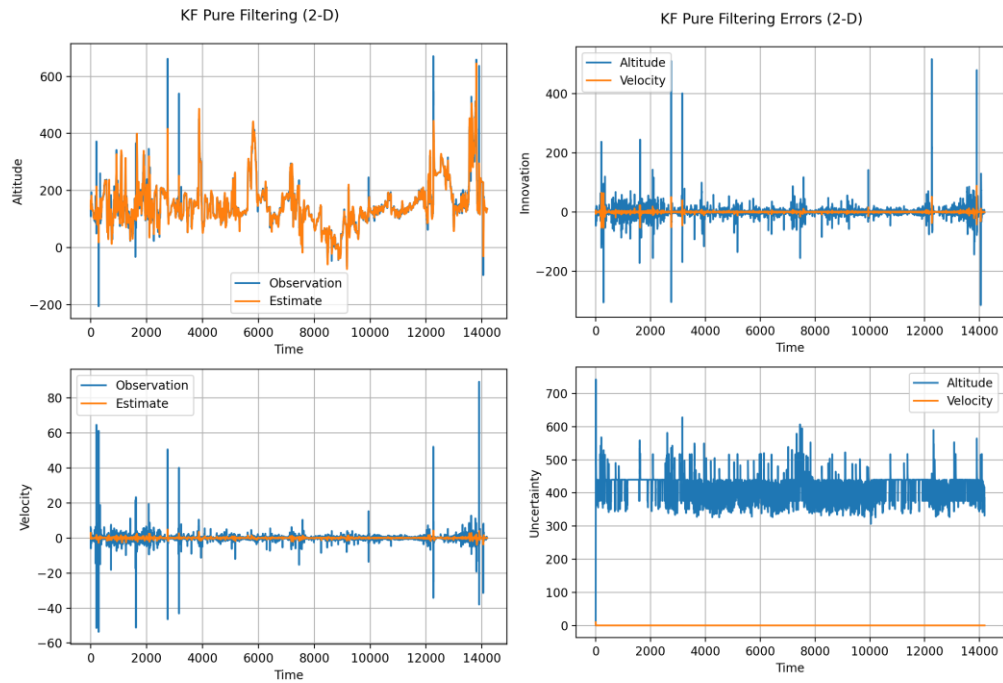
For the 3-D state estimation system:

$$F = \begin{bmatrix} 1 & \Delta t & \frac{\Delta t^2}{2} \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

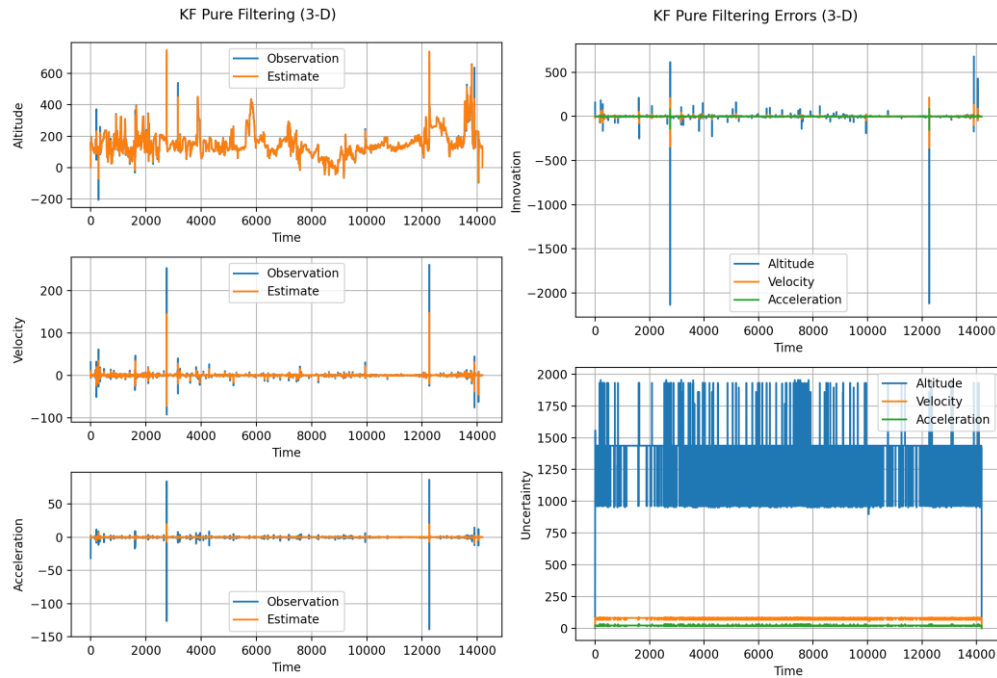
When developing the filters for each application, it was observed that outliers at the beginning of the trajectories would invalidate the initialization process. Therefore, initialization of state variables utilized the mean of the first set of elements, and the values in the covariance matrix were all set to values that would prevent the innovation and variance from becoming unstable. The values used are as follows:

State Variable	Assumed Covariance Value
Altitude	1
Velocity	10
Acceleration	20

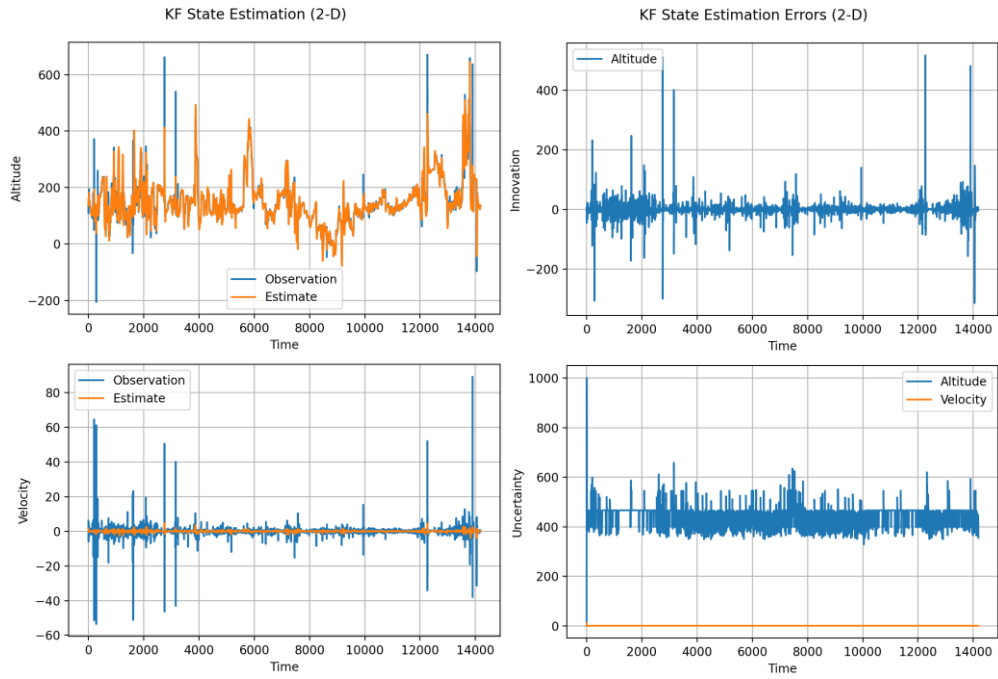
These values will be used as a baseline going forward. Other than the initialization, the Kalman Filter was tuned by adjusting the value of α to achieve stable covariance matrix and innovation values generally around ± 25 , except for a few outliers. This was done by keeping α small, usually around 10. The error plot below includes uncertainty values; these are the unprocessed values from the covariance matrix indicating the uncertainty in the estimates. To find the variance, the square root of the uncertainty is performed.



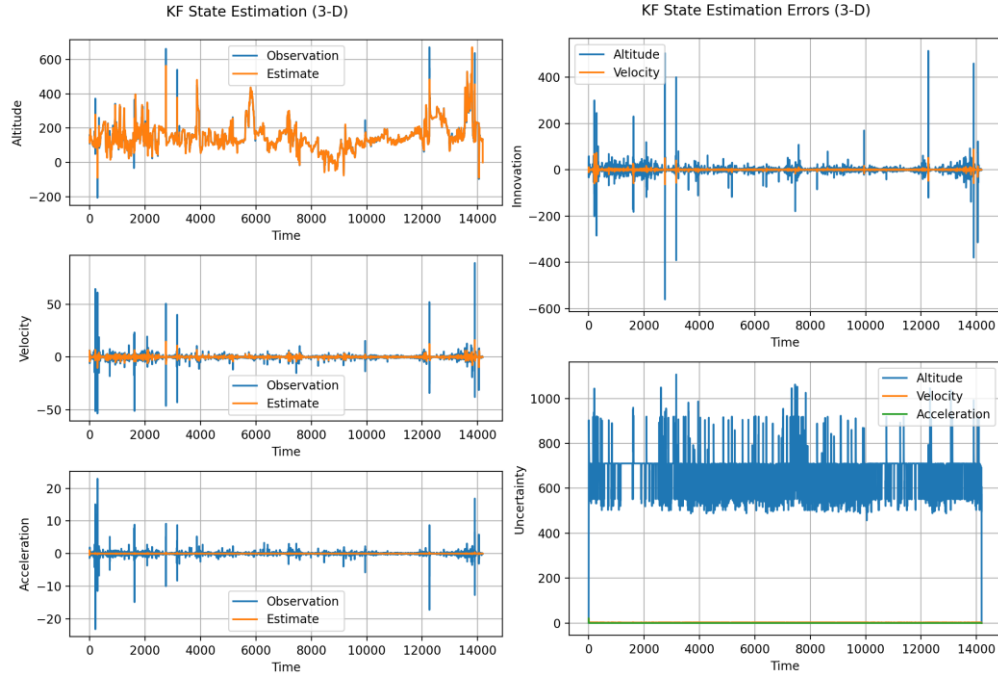
Run Time (s)	Innovation RMS			Average Variance	
	Altitude	Velocity	Total	Altitude	Velocity
1.2	19.5	2.1	19.7	20.4	0.6



Run Time (s)	Innovation RMS				Average Variance		
	Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration
1.2	27.7	5.8	2.2	28.4	36.9	8.9	4.6



Run Time (s)	Innovation RMS		Average Variance	
	Altitude		Altitude	Velocity
0.92	20.8		21.0	0.6



Run Time (s)	Innovation RMS			Average Variance		
	Altitude	Velocity	Total	Altitude	Velocity	Acceleration
0.8	16.4	2.3	16.5	25.9	1.4	0.1

Kalman Filters and Nonlinear Systems: The Extended Kalman Filter

From Haykin, the extended Kalman filter process is outlined as follows [6]:

State-space model

$$\begin{aligned}x_{k+1} &= f(k, x_k) + w_k \\ y_k &= h(k, x_k) + v_k\end{aligned}$$

Where, w_k and v_k are:

- Independent
- Zero-mean
- Normal
- Covariances of Q_k and R_k , respectively.

We define the parameters in the state-space model with Haykin's notation and processes [6]:

$$\begin{aligned}F_{k+1,k} &= \left. \frac{\partial f(k, x)}{\partial x} \right|_{x=x_k} \\ H_k &= \left. \frac{\partial h(k, x)}{\partial x} \right|_{x=x_k^-}\end{aligned}$$

Initialization:

$$\begin{aligned}\hat{x}_0 &= E[x_0] \\ P_0 &= E[(x_0 - E[x_0]) (x_0 - E[x_0])^T]\end{aligned}$$

The iterative portion, for each $k = 1, 2, 3, \dots$:

State estimate propagation

$$\hat{x}_k^- = f(k, \hat{x}_{k-1})$$

Error covariance propagation

$$P_k^- = F_{k,k-1} P_{k-1} F_{k,k-1}^T + Q_{k-1}$$

Kalman gain matrix

$$G_k = P_k^- H_k^T [H_k P_k^- H_k^T + R_k]^{-1}$$

State estimate update

$$\hat{x}_k = \hat{x}_k^- + G_k y_k - h(k, \hat{x}_k^-)$$

Error covariance update

$$P_k = (I - G_k H_k) P_k^-$$

For the 2-dimensional state-space model:

$$x_{k+1} = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}$$

We define $f(k, x_k)$ as follows:

$$f(k, x_k) = \begin{bmatrix} x_k + \Delta t v_k \\ v_k \end{bmatrix}$$

And $h(k, x_k)$ becomes:

$$\text{position only measurement} \rightarrow h(k, x_k) = x$$

$$\text{position and velocity only measurement} \rightarrow h(k, x_k) = \begin{bmatrix} x \\ v \end{bmatrix}$$

Now, defining F_k and H_k

$$F_k \text{ or } F_{k,k=1} = \frac{\partial f(k, x)}{\partial x} \Big|_{x=x_k} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial v} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial v} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

$$h(k, x_k) = x \rightarrow H_k = \frac{\partial h(k, x)}{\partial x} \Big|_{x=x_k} = \begin{bmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial v} \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

For a 3-dimensional case, we define the state space as follows:

$$x_{k+1} = \begin{bmatrix} \text{altitude} \\ \text{velocity} \\ \text{acceleration} \end{bmatrix}$$

Which allows us to define $f(k, x_k)$ as follows:

$$f(k, x_k) = \begin{bmatrix} x_k + \Delta t v_k + \frac{1}{2} \Delta t^2 a_k \\ v_k + a_k \Delta t \\ a_k \end{bmatrix}$$

And $h(k, x_k)$ becomes:

$$\text{position and velocity only measurement} \rightarrow h(k, x_k) = x$$

$$\text{measuring each state – space variable} \rightarrow h(k, x_k) = \begin{bmatrix} x \\ v \\ a \end{bmatrix}$$

Now, defining F_k and H_k

$$F_k \text{ or } F_{k,k=1} = \frac{\partial f(k, x)}{\partial x} \Big|_{x=x_k} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial v} & \frac{\partial f_1}{\partial a} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial v} & \frac{\partial f_2}{\partial a} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial v} & \frac{\partial f_3}{\partial a} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & \frac{1}{2} \Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

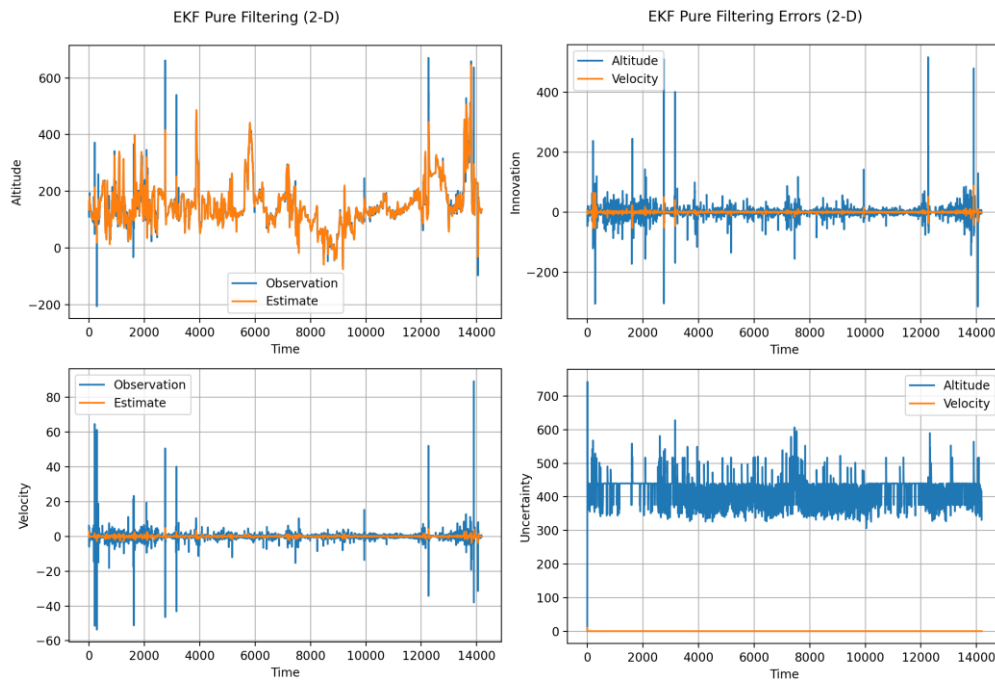
$$h(k, x_k) = \begin{bmatrix} x \\ v \end{bmatrix} \rightarrow H_k = \frac{\partial h(k, x)}{\partial x} \Big|_{x=x_k} = \begin{bmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial v} \\ \frac{\partial h_2}{\partial x} & \frac{\partial h_2}{\partial v} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$h(k, x_k) = \begin{bmatrix} x \\ v \\ a \end{bmatrix} \rightarrow H_k = \frac{\partial h(k, x)}{\partial x} \Big|_{x=x_k} = \begin{bmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial v} & \frac{\partial h_1}{\partial a} \\ \frac{\partial h_2}{\partial x} & \frac{\partial h_2}{\partial v} & \frac{\partial h_2}{\partial a} \\ \frac{\partial h_3}{\partial x} & \frac{\partial h_3}{\partial v} & \frac{\partial h_3}{\partial a} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

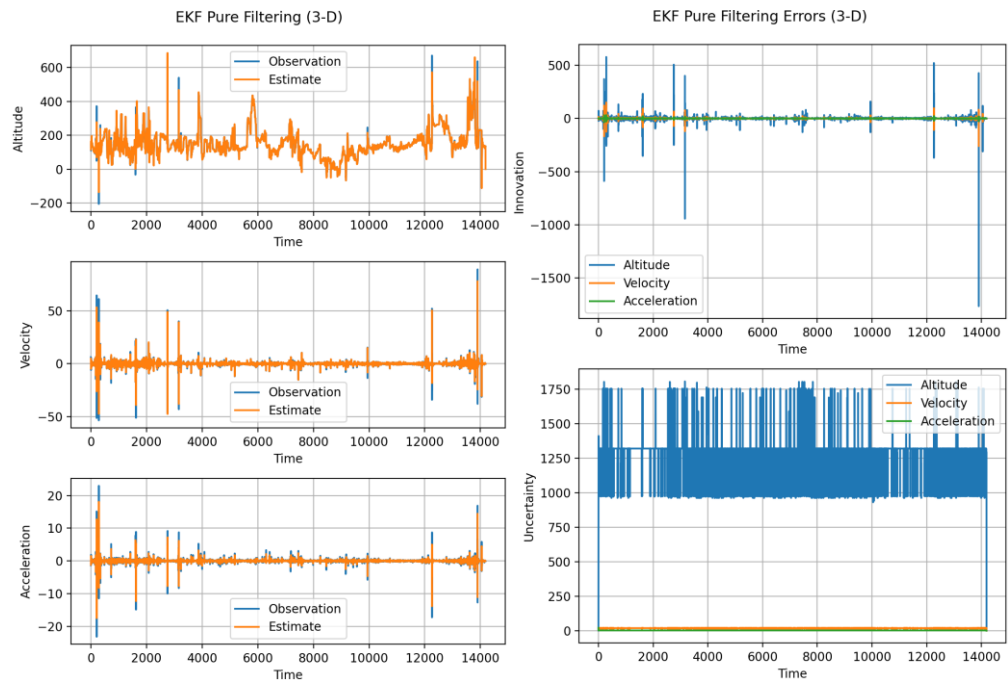
As done previously, the covariance matrix values were assumed to ensure stability. The values used are as follows.

State Variable	Assumed Covariance Value
Altitude	1
Velocity	10
Acceleration	20

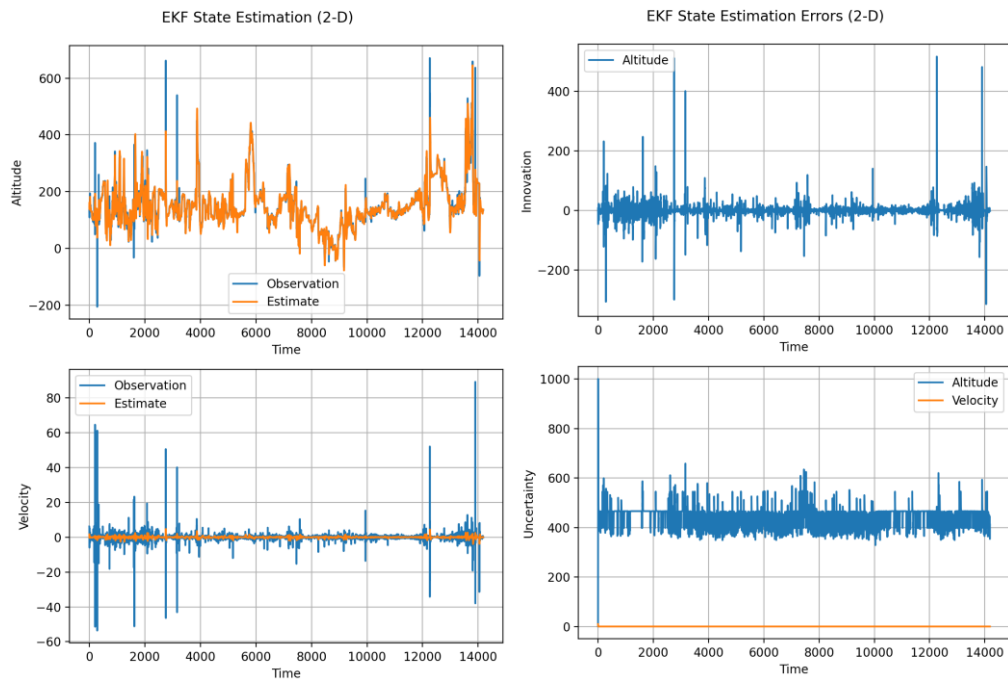
The extended Kalman Filter was also tuned by adjusting the value of α to achieve stable covariance matrix values and innovation values generally around ± 25 , except for a few outliers. The RMS innovation magnitude was approximately ± 25 . This was done by keeping α small, usually around 10.



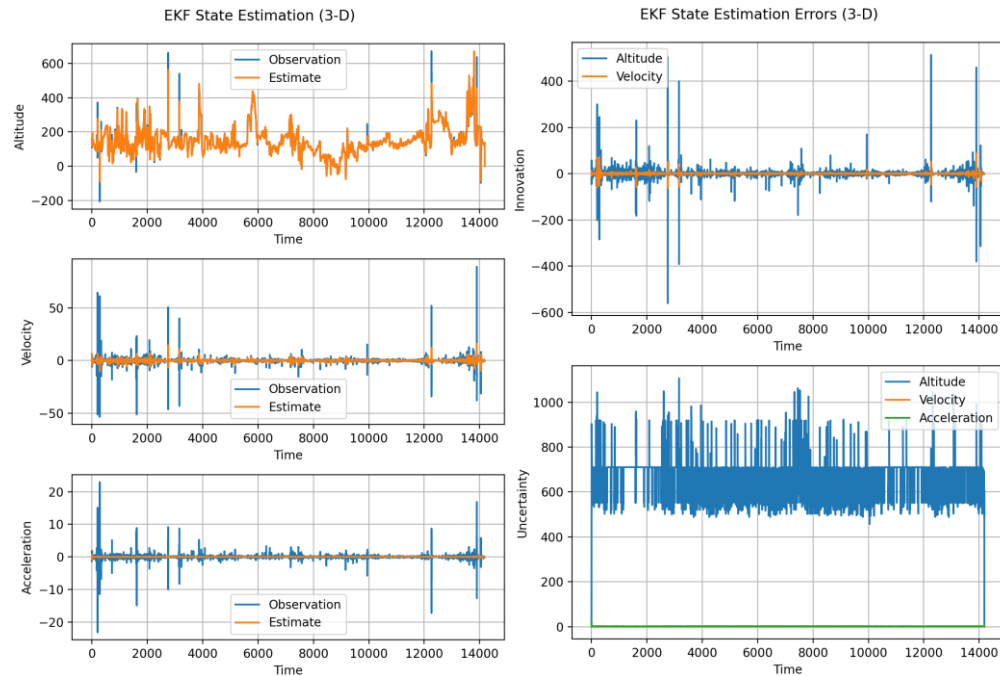
Run Time (s)	Innovation RMS			Average Variance	
	Altitude	Velocity	Total	Altitude	Velocity
1.2	19.5	2.1	19.7	20.4	0.65



Run Time (s)	Innovation RMS				Average Variance		
	Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration
0.8	22.9	4.6	0.9	23.4	35.6	4.2	1.1



Run Time (s)	Innovation RMS		Average Variance	
	Altitude		Altitude	Velocity
1.2	20.8		21.0	0.6



Run Time (s)	Innovation RMS			Average Variance		
	Altitude	Velocity	Total	Altitude	Velocity	Acceleration
1.4	16.4	2.3	16.5	25.9	1.4	0.1

Kalman Filters and Nonlinear Systems: The Unscented Kalman Filter

From Haykin, the Unscented Kalman filter state-space model [6]:

$$x_{k+1} = f(k, x_k) + w_k$$

$$y_k = h(k, x_k) + v_k$$

Where, w_k and v_k are:

- Independent
- Zero-mean
- Normal
- Covariances of Q_k and R_k , respectively.

Again, Haykin defines the following parameters in the spatespace model[6]:

$$F_{k+1,k} = \frac{\partial f(k, x)}{\partial x} \bigg|_{x=x_k}$$

$$H_k = \frac{\partial h(k, x)}{\partial x} \big|_{x=x_k^-}$$

Initialize using:

$$\hat{x}_0 = E[x_0]$$

$$P_0 = E[(x_0 - \hat{x}_0)(x_0 - \hat{x}_0)^T]$$

$$\hat{x}_0^a = E[x^a] = [\hat{x}_0^T \quad 0 \quad 0]^T$$

$$P_0^a = E[(x_0^a - \hat{x}_0^a)(x_0^a - \hat{x}_0^a)^T] = \begin{bmatrix} P_0 & 0 & 0 \\ 0 & R^v & 0 \\ 0 & 0 & R^n \end{bmatrix}$$

The iterative portion, for each $k = 1, 2, 3, \dots$:

Calculate the sigma points

$$\chi_{k-1}^a = \begin{bmatrix} \hat{x}_{k-1}^a & \hat{x}_{k-1}^a + \gamma \sqrt{P_{k-1}^a} & \hat{x}_{k-1}^a - \gamma \sqrt{P_{k-1}^a} \end{bmatrix}$$

Time update equations:

$$\chi_{k|k-1}^x = F(\chi_{k-1}^x, u_{k-1}, \chi_{k-1}^v)$$

$$\hat{x}_k^- = \sum_{i=1}^{2L} W_i^{(m)} \chi_{i,k|k-1}^x$$

$$P_k^- = \sum_{i=1}^{2L} W_i^{(c)} (\chi_{i,k|k-1}^x - \hat{x}_k^-)(\chi_{i,k|k-1}^x - \hat{x}_k^-)^T$$

$$\gamma_{k|k-1} = H(\chi_{k|k-1}^x, \chi_{k-1}^n)$$

$$\hat{y}_k^- = \sum_{i=0}^{2L} W_i^{(m)} \gamma_{i,k|k-1}$$

Measurement-update equations are:

$$P_{\tilde{y}_k \tilde{y}_k} = \sum_{i=0}^{2L} W_i^{(c)} (\gamma_{i,k|k-1} - \hat{y}_k^-)(\gamma_{i,k|k-1} - \hat{y}_k^-)^T$$

$$P_{x_k y_k} = \sum_{i=0}^{2L} W_i^{(c)} (\chi_{i,k|k-1}^x - \hat{x}_k^-)(\gamma_{i,k|k-1} - \hat{y}_k^-)^T$$

$$\kappa_k = P_{x_k y_k} P_{\tilde{y}_k \tilde{y}_k}^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + \kappa_k (y_k - \hat{y}_k^-)$$

$$P_k = P_k^- - \kappa_k P_{\tilde{y}_k \tilde{y}_k} \kappa_k^T$$

Where:

$$x^a = [x^T \quad v^T \quad n^T]^T$$

$$\chi^a = [(\chi^x)^T \quad (\chi^v)^T \quad (\chi^n)^T]^T$$

$$\Gamma = \sqrt{L + \lambda}$$

And other parameters are defined as:

$$W_0^{(m)} = \frac{\lambda}{L + \lambda}$$

$$W_0^{(c)} = \frac{\lambda}{L + \lambda} + 1 - \alpha^2 + \beta$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(L + \lambda)}, i = 1, \dots, 2L$$

Additionally, the Unscented Rauch-Tung-Striebel Smoother is defined as follows [7]:

$$G_k = P_k^f (P_k^-)^{-1}$$

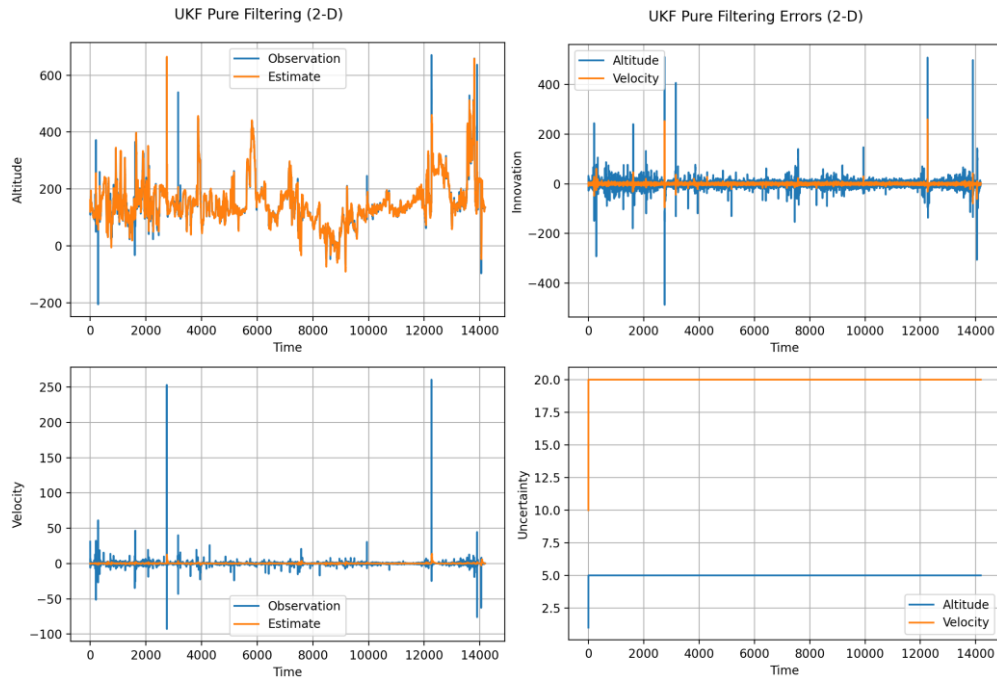
$$\hat{x}_k^{smooth} = \hat{x}_k + G_k (\hat{x}_{k+1}^{smooth} - \hat{x}_{k+1}^-)$$

$$P_k^{smooth} = P_k + G_k (P_{k+1}^{smooth} - P_{k+1}^-) G_k^T$$

As done previously, the covariance matrix values were assumed to ensure stability. The values used are as follows.

State Variable	Assumed Covariance Value
Altitude	1
Velocity	10
Acceleration	20

The unscented Kalman Filter has more tuning parameters than the Kalman and Extended Kalman filters, including tuning using α and β . The first parameter, α , was set at a baseline point where there were stable covariance matrix values and innovation values generally within ± 25 , except for a few outliers. The RMS innovation magnitude was approximately ± 25 . The second parameter, β , was set to 2 which is recommended for normally distributed data.

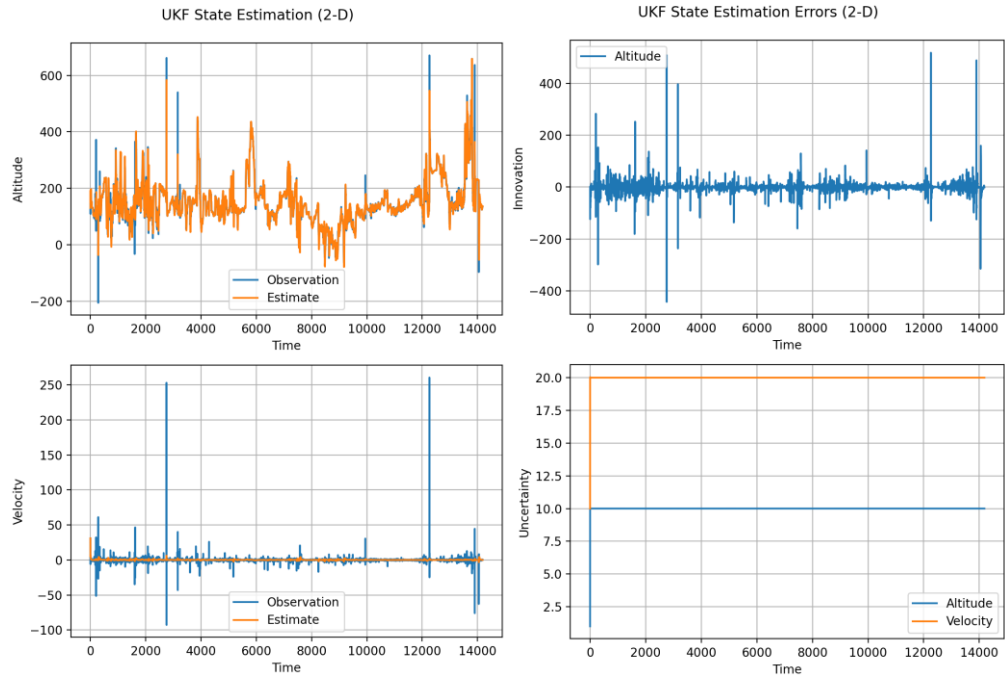


Run Time (s)	Innovation RMS			Average Variance	
	Altitude	Velocity	Total	Altitude	Velocity
13.9	19.4	4.4	19.9	2.2	4.5

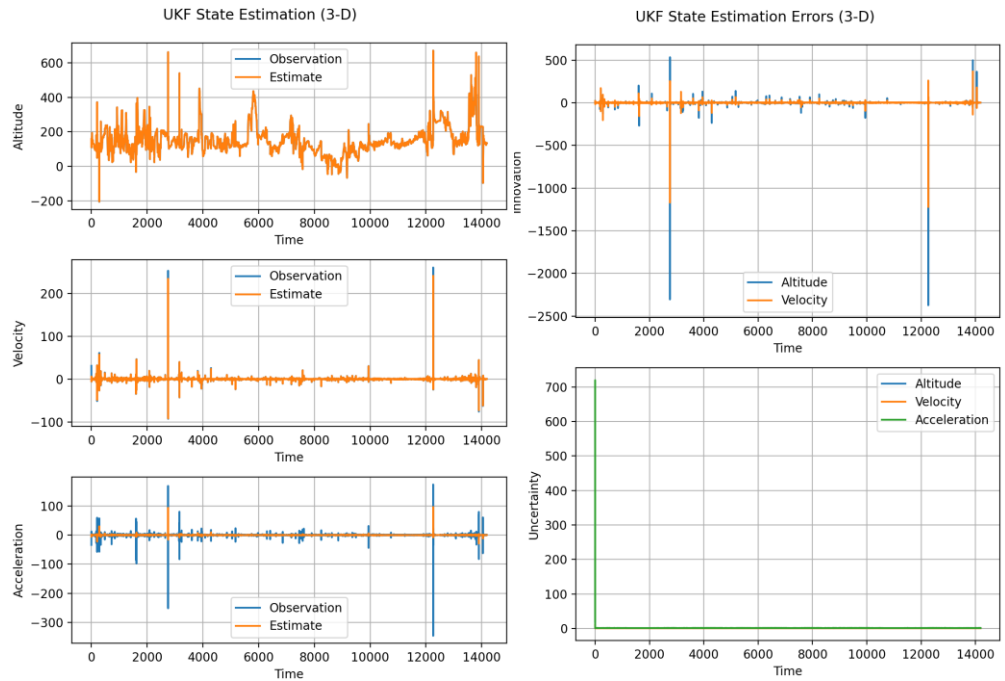


Run Time (s)	Innovation RMS				Average Variance		
	Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration

13.1	14.9	26.6	6.2	31.1	1	0.71	0.45
------	------	------	-----	------	---	------	------



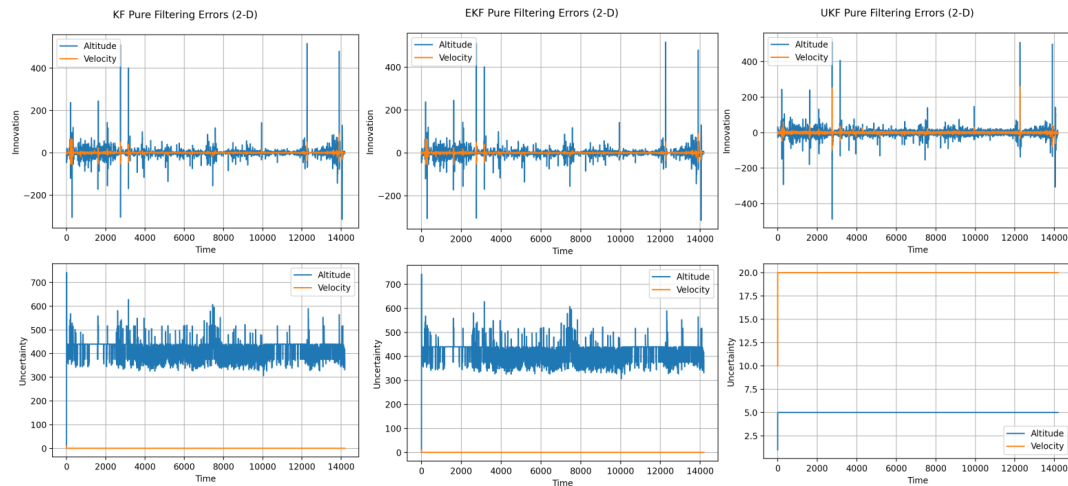
Run Time (s)	Innovation RMS	Average Variance	
	Altitude	Altitude	Velocity
10.8	18.7	3.2	4.5



Run Time (s)	Innovation RMS			Average Variance		
	Altitude	Velocity	Total	Altitude	Velocity	Acceleration
29.8	29.3	15.9	33.4	0.99	0.7	1.04

Comparison

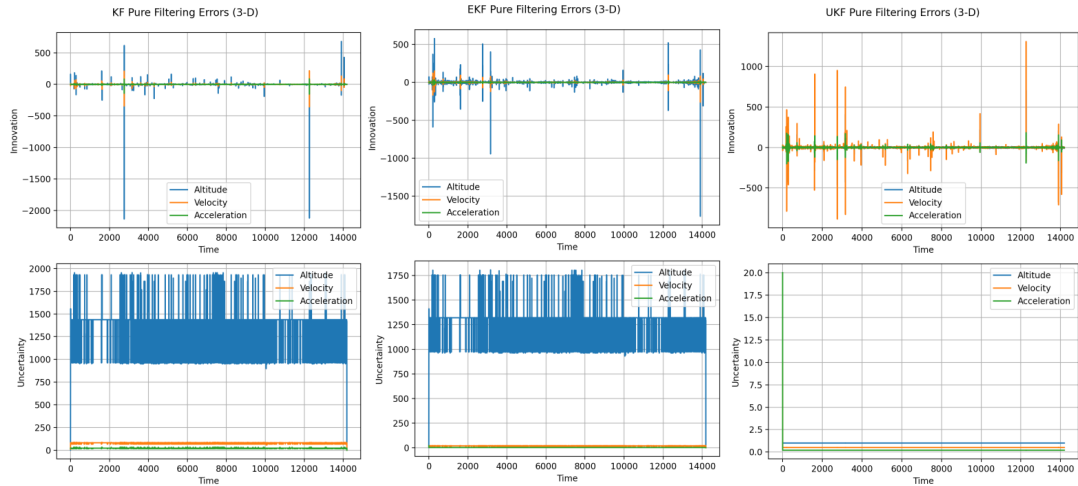
The previous section was intended to outline the baseline performance of each filter for both applications. To better understand the performance, the error plots and data are included below for each application.



For the 2-D pure filtering, it follows that the innovation values are all similar, with the KF and EKF being nearly identical. The uncertainty plots indicate that the KF and EKF have less confidence in their estimates than the UKF.

Filter Type	Run Time (s)	Innovation RMS			Average Variance	
		Altitude	Velocity	Total	Altitude	Velocity
KF	1.2	19.5	2.1	19.7	20.4	0.6
EKF	1.2	19.5	2.1	19.7	20.4	0.65
UKF	13.9	19.4	4.4	19.9	2.2	4.5

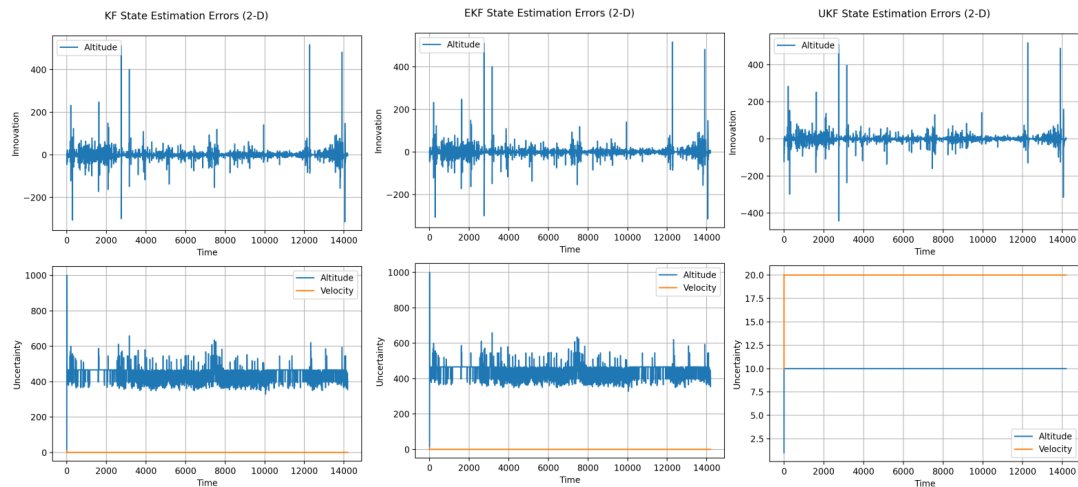
The RMS and average values above indicate similar behavior as the plots. Each filter has similar performance in terms of innovations, but the UKF outperforms with the variance. The run time does indicate that the UKF comes at a cost – nearly 10 times more compute time.



The KF and EKF have similar behaviors between the innovation and uncertainty plots. However, by examining the limits on the y-axis, it is apparent that the EKF captures the nonlinear dynamics of the 3-dimensional state-space and outperforms the KF. The UKF has even better visual performance than the EKF, with high confidence in the estimates and even narrower innovation bounds.

Filter Type	Run Time (s)	Innovation RMS				Average Variance		
		Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration
KF	1.2	27.7	5.8	2.2	28.4	36.9	8.9	4.6
EKF	0.8	22.9	4.6	0.9	23.4	35.6	4.2	1.1
UKF	13.1	14.9	26.6	6.2	31.1	1	0.71	0.45

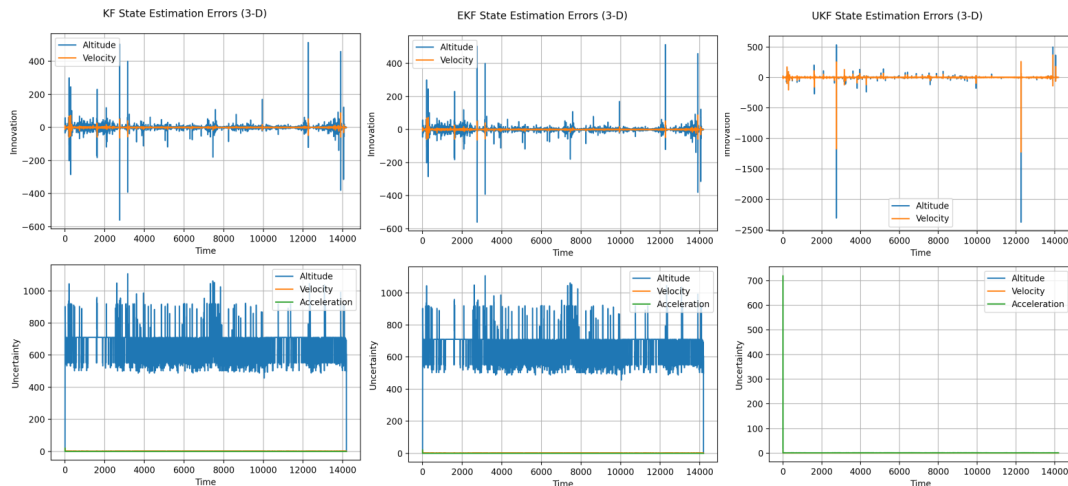
The RMS and average values concur with the visuals, somewhat. The UKF is the lowest performing for innovation, mostly due to velocity and acceleration, but has the best performance with the variance. Again, the compute time of the UKF is over 10 times more than the KF and EKF.



The KF and EKF are almost identical, which tracks since the 2-dimensional state-space is linear. Each innovation plot also seems identical, with only minor differences in the UKF only. For uncertainty, the confidence in altitude the best in the UKF, but the KF and EKF perform better with velocity.

Filter Type	Run Time (s)	Innovation RMS			Average Variance	
		Altitude	Velocity	Total	Altitude	Velocity
KF	0.92	20.8	21.0	0.6		
EKF	1.2	20.8	21.0	0.6		
UKF	10.8	18.7	3.2	4.5		

As expected, the innovation values are all similar, with the KF and EKF being identical. For the variance, the UKF performs better with altitude whereas the KF and EKF have better confidence with velocity.



The 3-dimensional state-space system should be nonlinear in practice. Since the KF and EKF don't include the nonlinearity associated with measuring the acceleration state variable, they become linear and yield identical plots. The UKF seems to have a higher innovation error than the KF and EKF, but seems to have higher confidence.

Filter Type	Run Time (s)	Innovation RMS			Average Variance		
		Altitude	Velocity	Total	Altitude	Velocity	Acceleration
KF	0.8	16.4	2.3	16.5	25.9	1.4	0.1
EKF	1.4	16.4	2.3	16.5	25.9	1.4	0.1
UKF	29.8	29.3	15.9	33.4	0.99	0.7	1.04

The mean values confirm that the KF and EKF behave identically due to the missing nonlinearity variables in the measure-space. The results indicate a tradeoff between compute time and innovation error versus uncertainty. The KF and EKF are quick and contain smaller innovation errors, but the UKF has better confidence.

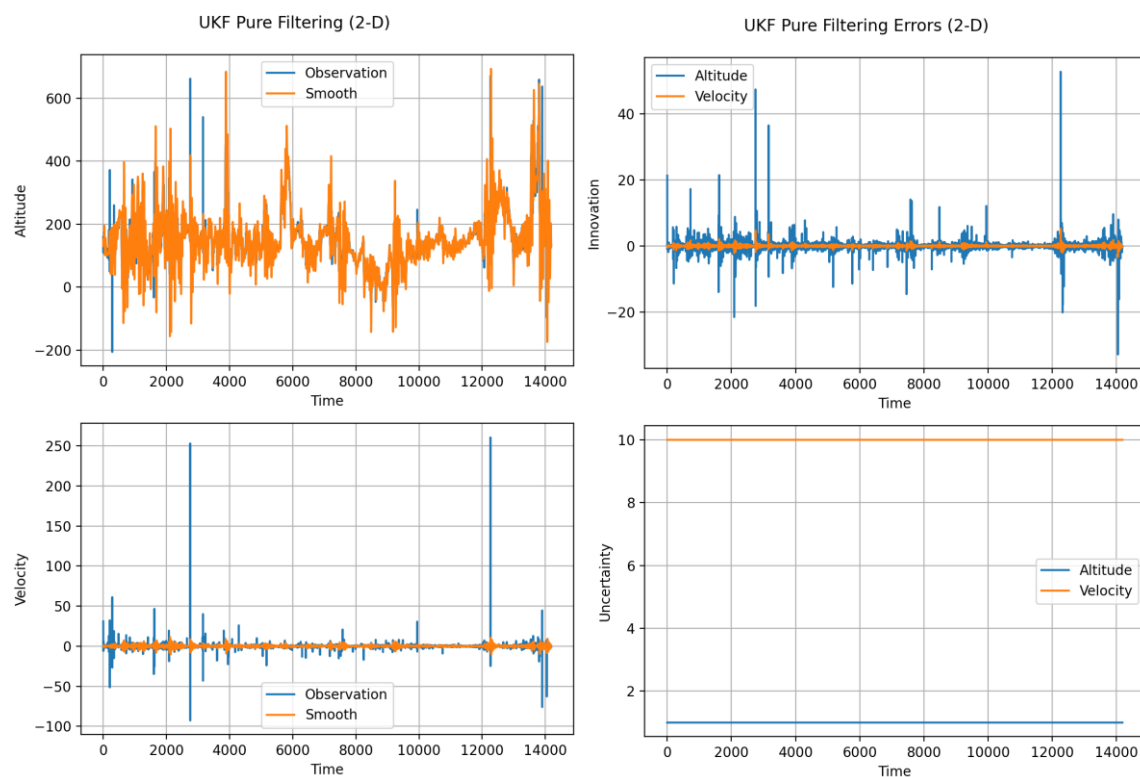
Overall, there are tradeoffs to be considered with each filter. Going forward, the 2-D pure filtering will use the UKF, the 3-D pure filtering the EKF, and the 2-D state estimation the KF. It could be argued that the UKF performed better than the EKF for the 3-D pure filtering. The decision to use the EKF for 3-D pure filtering was made to provide the opportunity to tune and test each filter at least once in the next sections.

Tuning

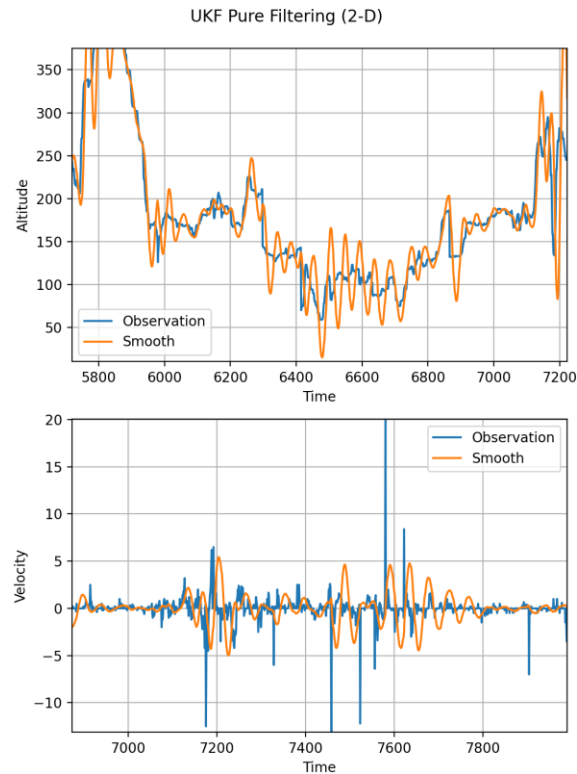
The tuning will be done manually. Mostly, it will change the α values in all filters to balance the variance with smoother estimates. Since the altitude data in the current trajectory deviates between -200 and 600, we seek to maintain a deviation of 30 to 40 to ensure confidence in the estimates. The unscented smoother will be applied to the tuned UKF to determine its efficacy.

2-D Filtering: UKF

By applying the unscented smoother outlined in the UKF baseline section and changing the α value from $1e-6$ to $1e-1$, the following results are gathered for the smooth UKF for the pure filtering in 2-dimensions.



We maintain innovation and uncertainty values all within our target ranges. With over 14,000 datapoints in the trajectory, it can be difficult to determine if the UKF is actually smoothing the estimates. Magnifying the visual, we get the following plots.



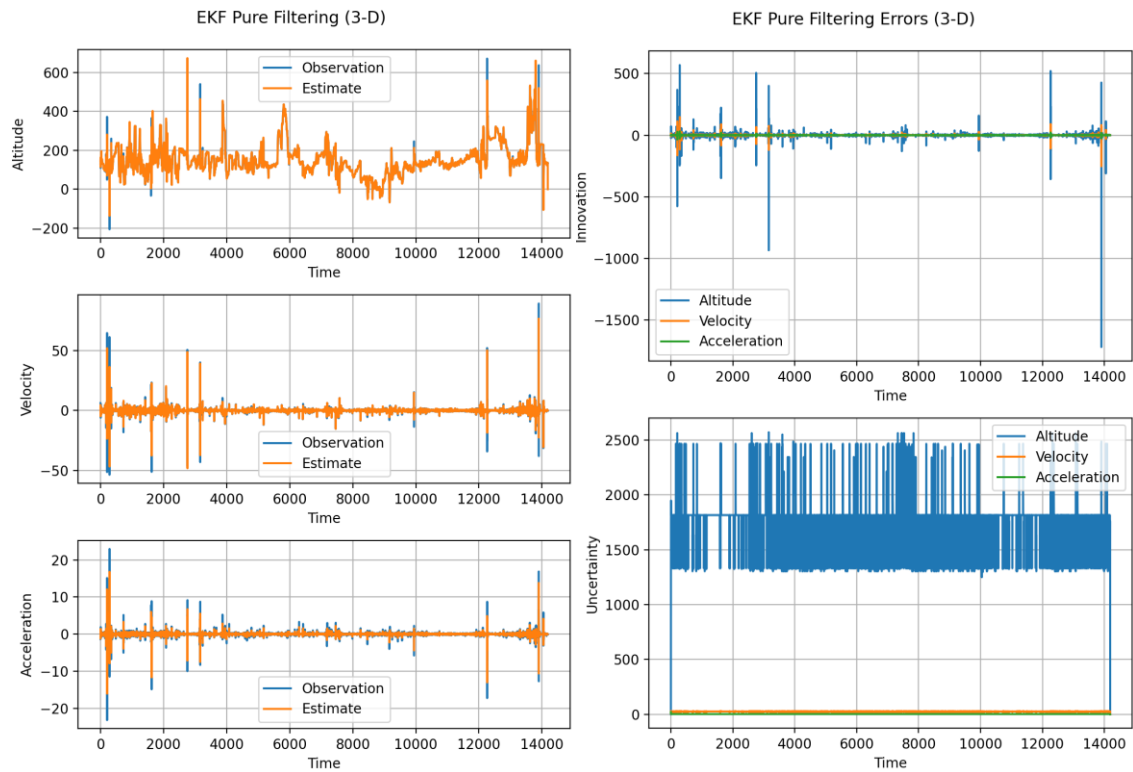
The visuals that magnify the estimates indicate that the smoother is working. There is still room for improvement to simplify the noise, since it is apparent that the estimate is oscillating about the noisy measurement.

Case	Run Time (s)	Innovation RMS			Average Variance	
		Altitude	Velocity	Total	Altitude	Velocity
Baseline	13.9	19.4	4.4	19.9	2.2	4.5
Tuned	11.2	1.7	0.3	11.7	1.0	3.2

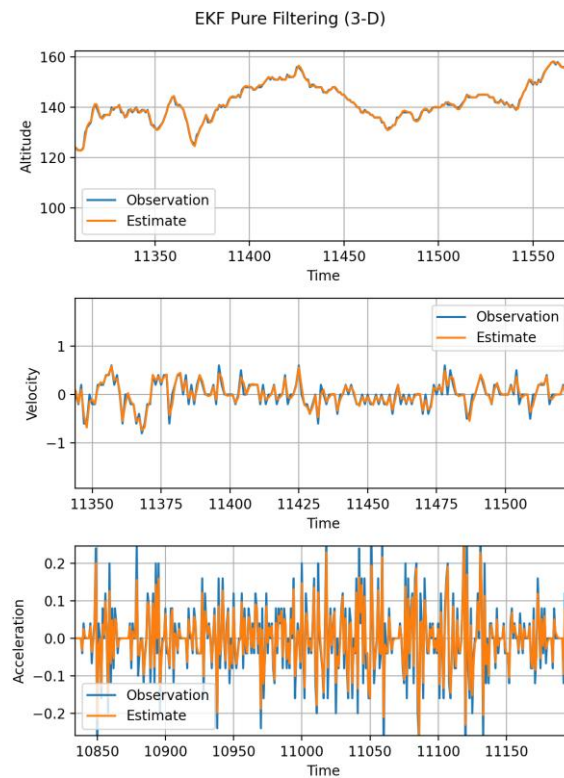
The tuned filter actually performs better in every metric than the baseline. This indicates that further tuning could be done to smooth the estimate, even if it meant a tradeoff between innovation and variance.

3-D Filtering: EKF

The EKF is tuned for the 3-D filtering application. The main tuning method is varying α value from 10 to 15, which leads to a tradeoff between variance and innovation. The following results are gathered for the smooth EKF for the pure filtering in 3-dimensions.



Without magnifying the results, it is difficult to determine if any smoothing occurs. The following plot provides more insight.



It is apparent that minimal smoothing occurs. The EKF struggles to make the noisy data more interpretable while preserving confidence in the estimates.

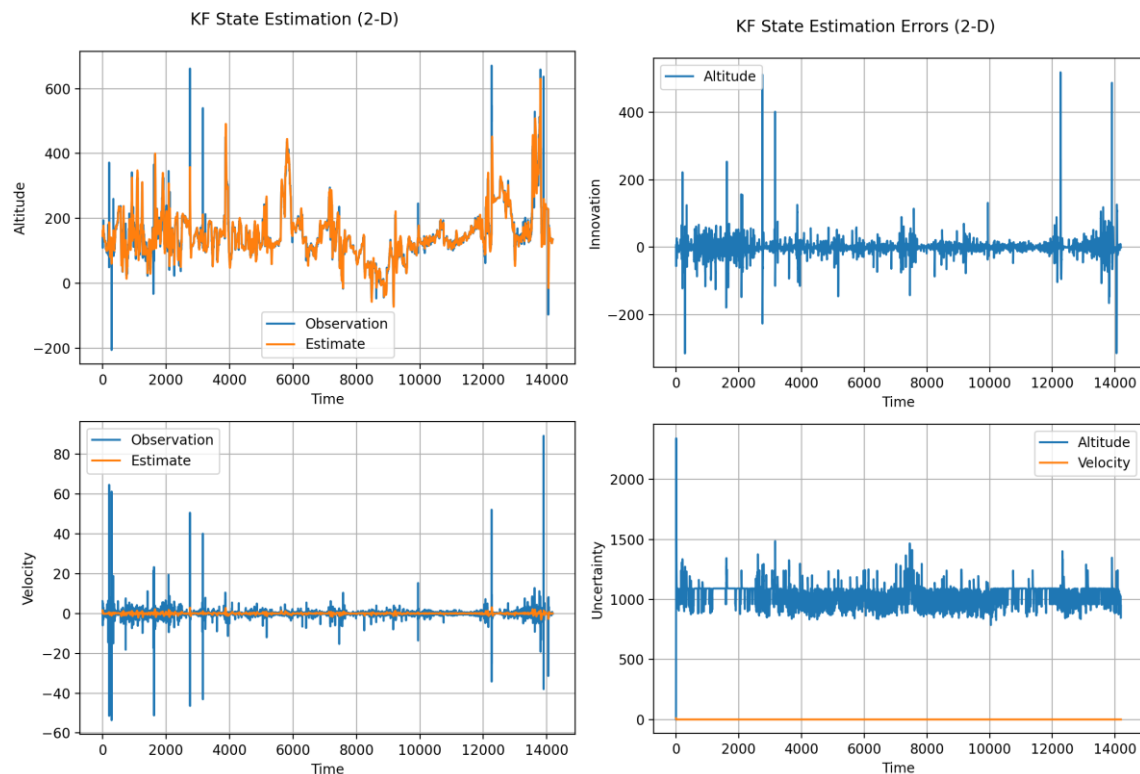
Case	Run Time (s)	Innovation RMS				Average Variance		
		Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration
Baseline	0.8	22.9	4.6	0.9	23.4	35.6	4.2	1.1
Tuned	0.8	22.5	4.5	0.9	22.9	41.7	5.0	1.3

While the innovation improves for the tuned model, the variance continues to increase past the desired range of 40 for the altitude. This indicates that the EKF doesn't meet the desired quantitative and qualitative metrics for innovation and variance, along with smoothing, respectively.

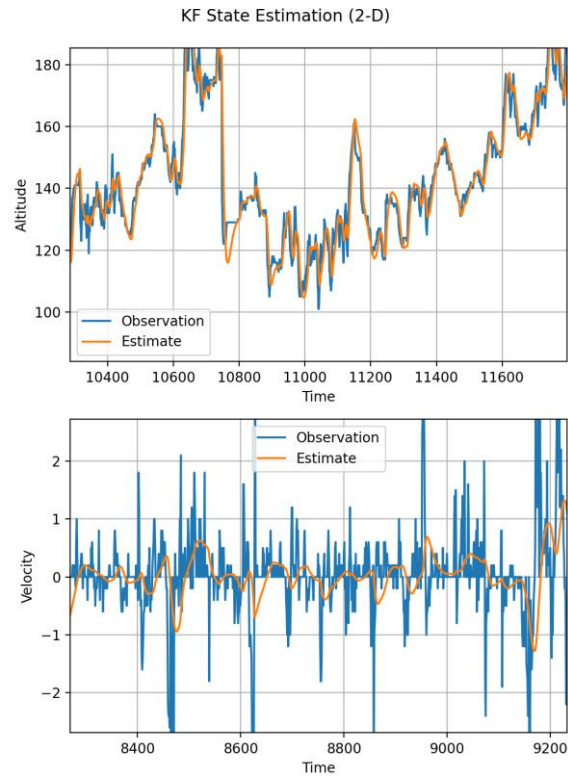
To improve performance for this application, the UKF should have been used. The KF is anticipated to have lower performance than the EKF due to the state-space consisting of a nonlinear system.

2-D State Estimation: KF

The KF is tuned for the 2-D state estimation application. The main tuning method involves varying α value from 10 to 30, which leads to a tradeoff between variance and innovation. The following results are gathered for the smooth KF for the pure filtering in 2-dimensions.



The results look promising. It is apparent that smoothing occurs for the altitude estimate. The uncertainty appears to hover around 1000, which will translate to a variance of 33. To determine the performance of the smoothing, the following plot magnifies the estimates.



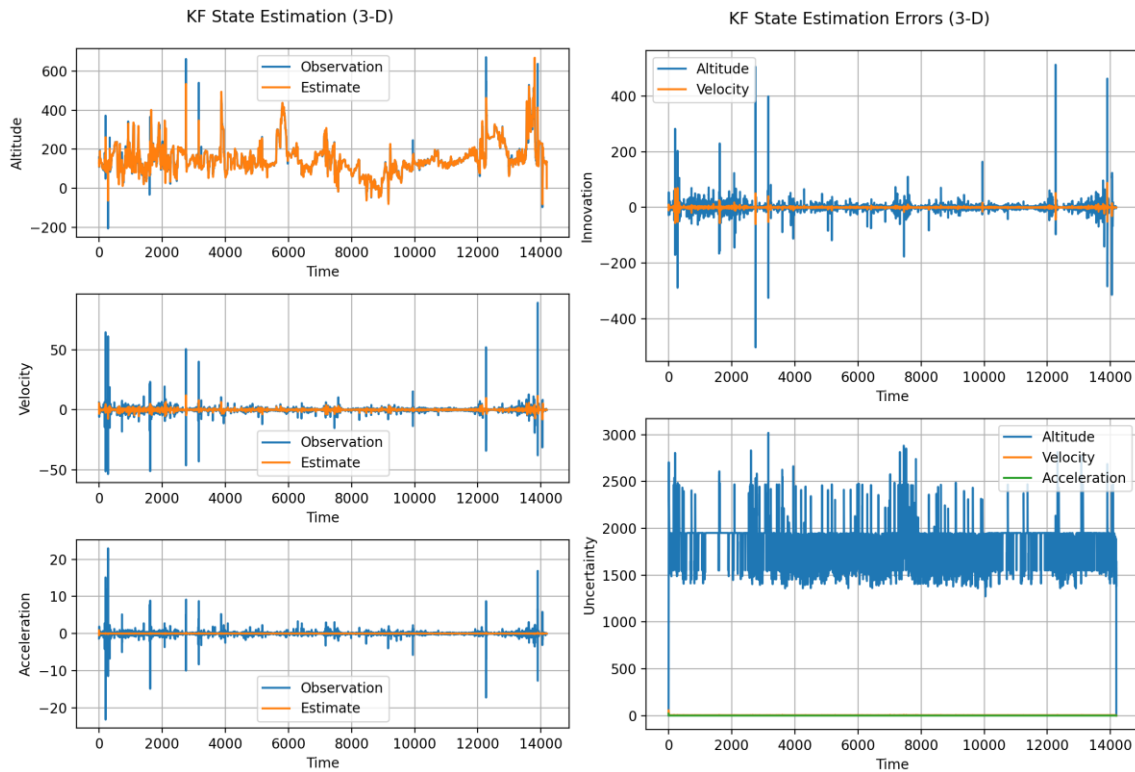
The smoothing performance appears to be qualitatively good. The estimates appear to have reduced the noise by providing smooth curves.

Case	Run Time (s)	Innovation RMS	Average Variance	
		Altitude	Altitude	Velocity
Baseline	0.92	20.8	21.0	0.6
Tuned	0.71	23.3	32.2	0.7

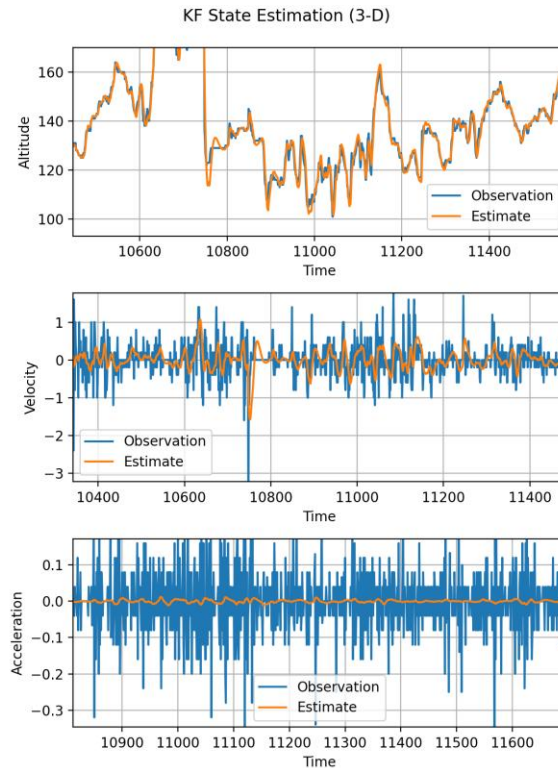
The innovation RMS and average variance both look promising. All values increased, but they comply with realistic expectations given the magnitude of our measurements.

3-D State Estimation: KF

The KF is tuned for the 3-D state estimation application. Similar to the 2-D example, the main tuning method involves varying α value from 10 to 30, which leads to a tradeoff between variance and innovation. The following results are gathered for the smooth EKF for the pure filtering in 3-dimensions.



The estimates appear to parse through the noisy measurements to create smooth plots. However, the uncertainty in the altitude appears to hover around 2000, which may exceed the realistic value for the dataset. To learn more about the level of smoothing that occurs, we magnify the estimates in the plot below.



The filter does an excellent job at parsing through the noise. If the error metrics indicate a lack of confidence in the estimate, there does appear to be some allowed margin to tradeoff the smoothing to increase performance.

Case	Run Time (s)	Innovation RMS			Average Variance		
		Altitude	Velocity	Total	Altitude	Velocity	Acceleration
Baseline	0.8	16.4	2.3	16.5	25.9	1.4	0.1
Tuned	0.55	17.3	2.3	17.5	42.9	2.0	0.1

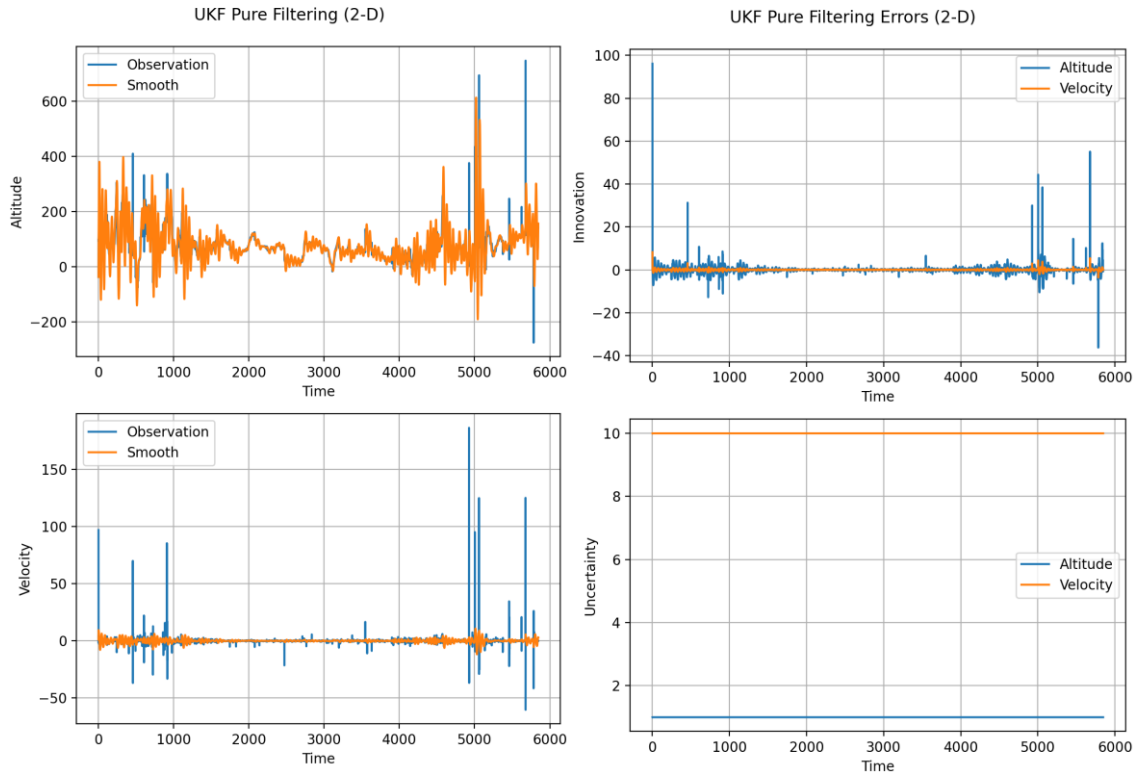
As expected, the variance does exceed the maximum desired value of 40 for altitude. To maintain quantitative performance metrics, it is recommended that α needs to be between 10 and 30.

Testing

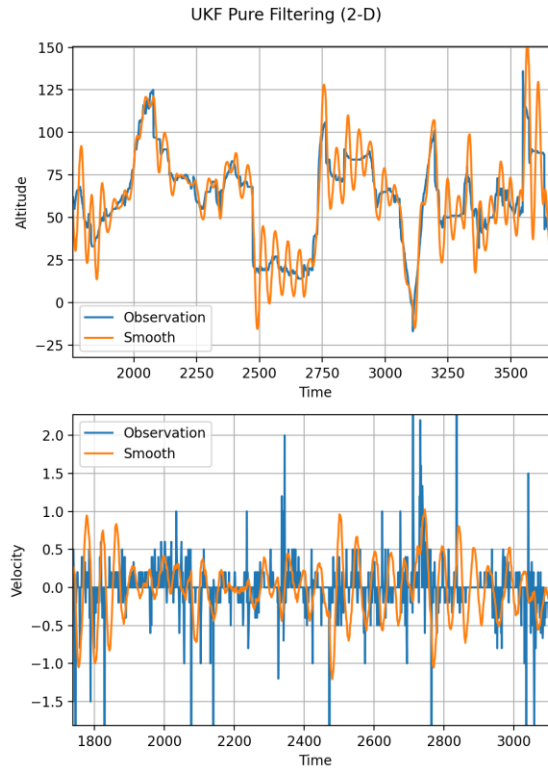
In this section, we apply a separate trajectory for each to further test the performance of the filters. The new trajectory is taken from the Microsoft GeoLife and was chosen since it was the second largest accessed with approximately 6000 datapoints. The goal is to determine whether performance increases, decreases, or is maintained.

2-D Filtering:

The tune UKF with the unscented smoother was used on the test trajectory for the pure filtering application in 2-dimensions. The results are displayed below.



The visuals indicate that tuning occurs at a macro-level, since the spikes in altitude and velocity are damped by the smooth UKF estimate. The errors still comply with the targeted ranges. To determine the level of smoothing, the estimate is magnified below.



The visuals indicate the same oscillatory behavior for the UKF that occurred in testing.

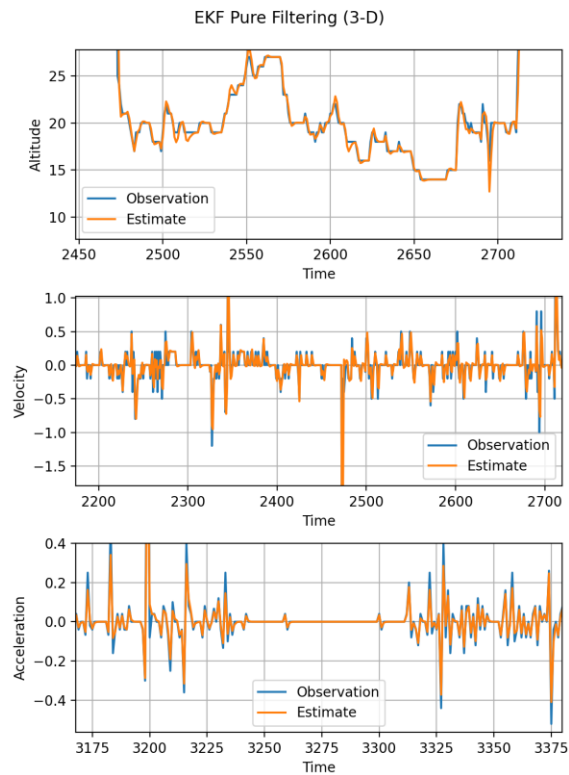
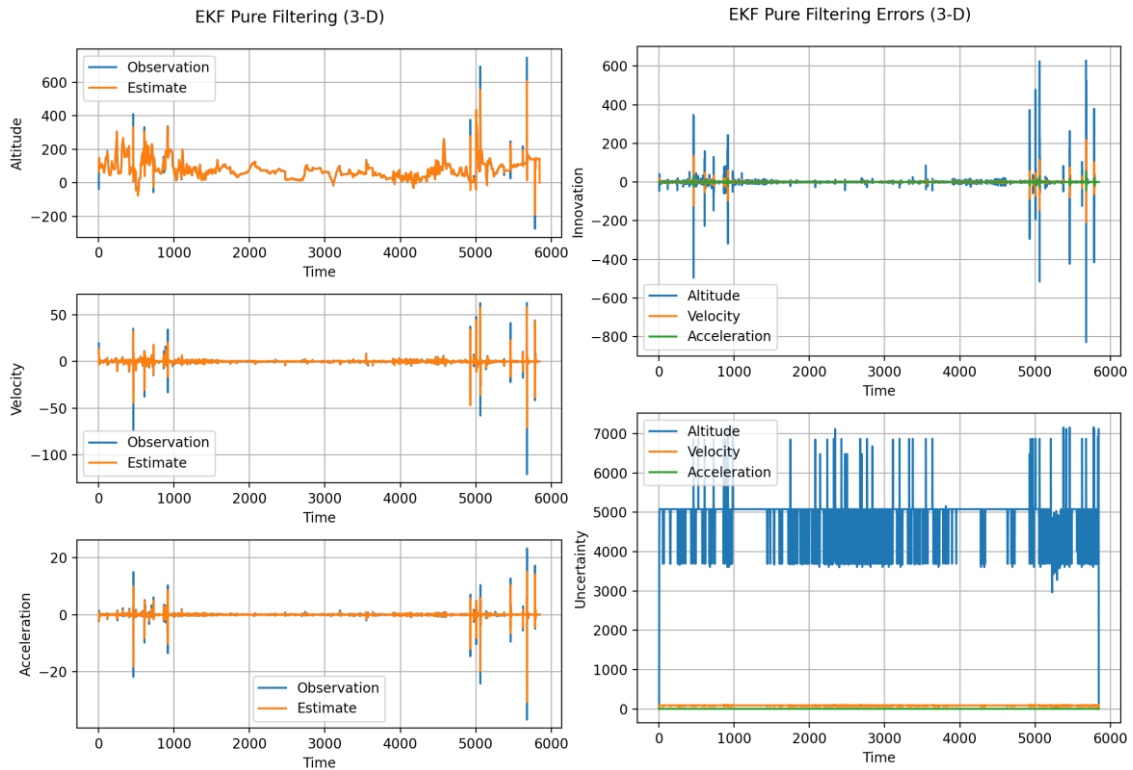
Trajectory	Run Time (s)	Innovation RMS			Average Variance	
		Altitude	Velocity	Total	Altitude	Velocity
Train	11.2	1.7	0.3	1.7	1.0	3.2
Test	5.4	2.4	0.3	2.4	1.0	3.2

Overall, the performance decreased slightly in terms of innovation. It was interesting to find that the variance did not change.

Overall, the performance of the UKF may indicate oscillatory behavior that may exceed the amplitude of the data, specifically the altitude measurement, but this may be preferred in situations where high levels of confidence are required.

3-D Filtering:

The tuned EKF for the 3-D filtering application is applied to the testing trajectory. The results in included below.



The results indicate some filtering occurs, specifically the outliers. The magnitudes of the uncertainties appear to increase past the targeted range. Then, the magnified visuals of the

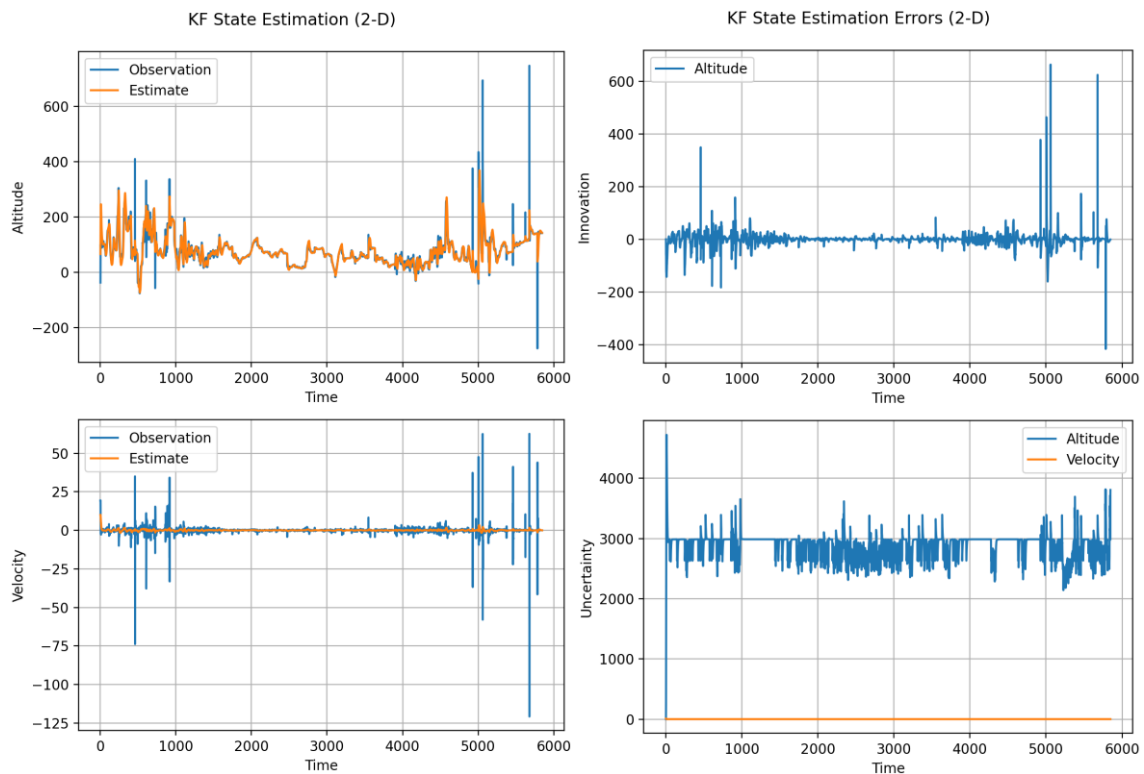
estimates show minimal smoothing for altitude and velocity, but some indications with acceleration.

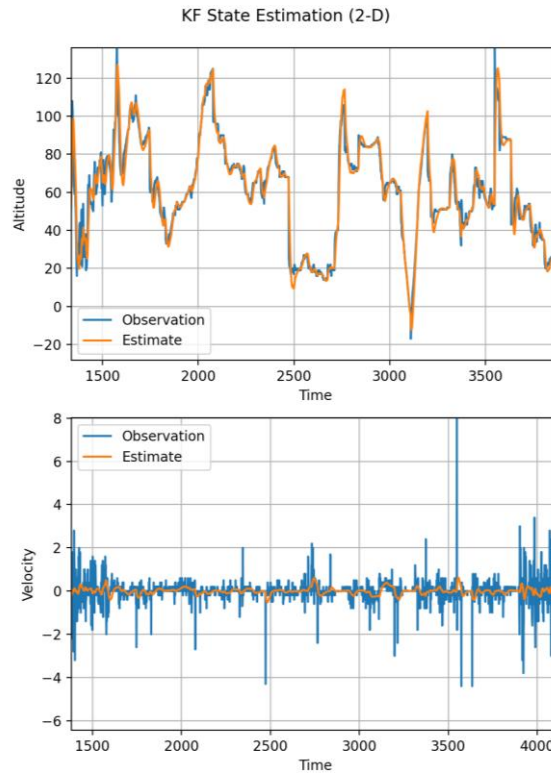
Trajectory	Run Time (s)	Innovation RMS				Average Variance		
		Altitude	Velocity	Acceleration	Total	Altitude	Velocity	Acceleration
Train	0.8	22.5	4.5	0.9	22.9	41.7	5.0	1.3
Test	0.4	27.5	6.8	1.5	28.4	70.1	9.2	1.7

The RMS and average errors confirm the indications from the error plot. The confidence in the filtered estimates decreases, along with the performance of the innovations.

2-D State Estimation:

The tuned KF for the 2-D state estimation application is applied to the testing trajectory. The results are as follows.





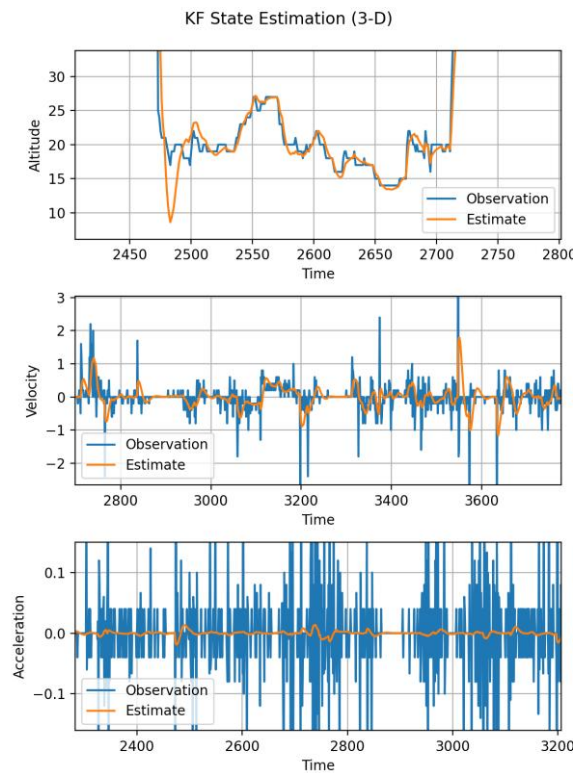
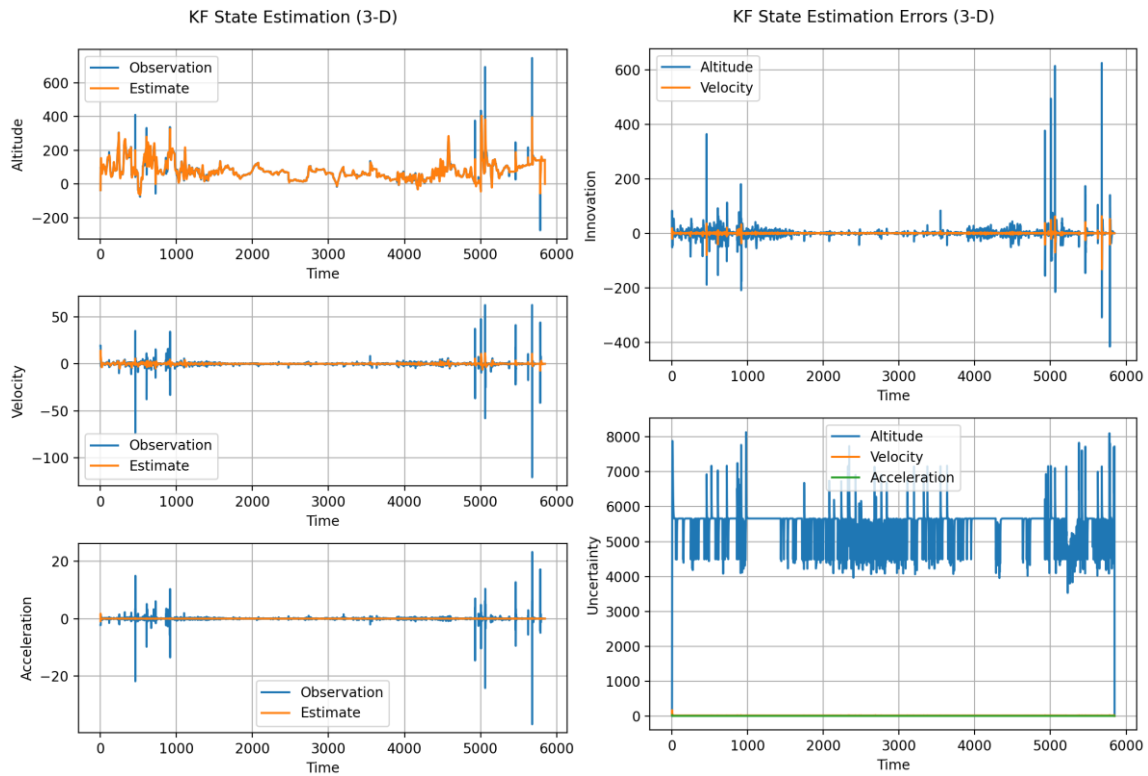
The results look promising, other than the increase in uncertainty, which surpasses the targeted range of 1000-2000. The KF appears to effectively filter the data by smoothing the estimate plots.

Trajectory	Run Time (s)	Innovation RMS	Average Variance	
		Altitude	Altitude	Velocity
Tuned	0.71	23.3	32.2	0.7
Test	0.34	27.3	53.6	1.15

The RMS and average values confirm our previous conclusions. The errors increase past our targeted values.

3-D State Estimation:

The tuned KF for the 3-D state estimation application is applied to the testing trajectory. The results are as follows.



While the KF appears to filter the data, it is apparent that the confidence in the estimate decreases past our desired values with testing trajectory.

Trajectory	Run Time (s)	Innovation RMS			Average Variance		
		Altitude	Velocity	Total	Altitude	Velocity	Acceleration
Train	0.55	17.3	2.3	17.5	42.9	2.0	0.1
Test	0.26	22.9	3.5	23.1	73.7	3.4	0.1

As expected, the errors KF for 3-D state estimation in the testing trajectory surpass realistic values, particularly the altitude average variance of 73.7.

Conclusion

The training results provided interesting insights into the performance of each filter. For pure filtering applications in 2-dimensions, the UKF showed excellent performance for providing low innovation values and high confidence in the estimates, but at the cost of drastically more compute power. The EKF underperformed in the 3-dimensional pure filtering application, with high innovation values, low confidence, and minimal qualitative smoothing in the estimates. For 2-D and 3-D state estimation, KF and EKF results were identical, indicating a linear state-space. Intuitively, this is a logical conclusion since neither application introduced acceleration to the state-space. The KF was applied to both applications and received similar results. The variance was towards the upper end of the acceptable range, but the filter performed well.

The testing results indicated equivalent or lower performance than the training results. The discrepancy can be attributed to multiple factors. It is possible that in tuning the filters, they were trained to overfit the training trajectory. Another possibility is the type of datapoints in the trajectory. The datapoints represent different modes of transportation such as walking, cycling, buses, and more. The different transportation modes could affect the noise of the data, which would require updates to the filters. To increase performance, parsing the trajectories could provide homogenous data that complies with the assumptions in the filter, such as the noise from the measurements.

Overall, the goal of applying the Kalman filter to the Microsoft GeoLife dataset was achieved. The major takeaway for the performance of Kalman filters is that none of them is particularly superior, but they each have their own application. The importance of preparing the dataset was also a takeaway after reviewing the discrepancies from testing and training trajectories.

Bibliography

- [1] *Understanding Kalman filters*. MATLAB.
<https://www.mathworks.com/videos/series/understanding-kalman-filters.html>
- [2] Welch, G., Bishop, G., & Department of Computer Science, University of North Carolina at Chapel Hill. (2006). *An introduction to the Kalman filter* (TR 95-041).
https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf
- [3] Pei, Y., Biswas, S., Fussell, D. S., & Pingali, K. (2017, October 9). *An elementary introduction to Kalman filtering*. arXiv.org. <https://arxiv.org/abs/1710.04055>
- [4] Oksendal, B. (2013). *Stochastic differential equations: An Introduction with Applications*. Springer Science & Business Media
- [5] Zheng, Y., Fu, H., Xie, X., Ma, W., & Li, Q. (2023, April 5). *Geolife GPS trajectory dataset - User Guide - Microsoft Research*. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/geolife-gps-trajectory-dataset-user-guide/>
- [6] Haykin, S. (Ed.). (2001). *KALMAN FILTERING AND NEURAL NETWORKS*. John Wiley & Sons, Inc.
https://cdn.preterhuman.net/texts/science_and_technology/artificial_intelligence/Kalman%20Filtering%20and%20Neural%20Networks%20-%20Simon%20Haykin.pdf
- [7] *Unscented Rauch--Tung--Striebel smoother*. (2008, April 1). IEEE Journals & Magazine | IEEE Xplore. <https://ieeexplore.ieee.org/document/4484208>